

TRABAJO DE FIN DE CICLO · 2.º DAM

AUTOCICLO

Sistema de Gestión Multiplataforma
para Desguace de Vehículos

ALUMNO

CICLO

CENTRO

CURSO

Yalil Musa Talhaoui

2.º DAM

IES P. Hermenegildo
Lanz

2025 / 2026

DAM — Multiplatafor-
ma Granada

Mayo 2026

ENTREGA · 19 Mayo 2026

Índice de contenidos

1	AutoCiclo	5
1.1	Sistema de Gestión Multiplataforma para Desguace de Vehículos	5
2	Índice	6
3	1. Introducción	7
3.1	1.1 Contexto y motivación	7
3.2	1.2 Descripción general	7
3.3	1.3 Objetivos	8
3.3.1	Objetivo general	8
3.3.2	Objetivos específicos	8
3.4	1.4 Alcance	9
3.5	1.5 Metodología	9
4	2. Análisis del sistema	10
4.1	2.1 Identificación de actores	10
4.2	2.2 Requisitos funcionales	10
4.2.1	RF-01 — Gestión de usuarios y autenticación	10
4.2.2	RF-02 — Catálogo de vehículos y piezas	10
4.2.3	RF-03 — Sistema de solicitudes de presupuesto	11
4.2.4	RF-04 — Sistema de negociación multi-ronda	11
4.2.5	RF-05 — Pagos con Stripe	11
4.2.6	RF-06 — Gestión de stock	12
4.2.7	RF-07 — Códigos QR	12
4.2.8	RF-08 — Notificaciones	12
4.2.9	RF-09 — Integración Odoo	12
4.2.10	RF-10 — Mensajería asíncrona (RabbitMQ)	12
4.3	2.3 Requisitos no funcionales	12
4.4	2.4 Casos de uso	14
4.4.1	CU-01 — Solicitar presupuesto (Cliente)	14
4.4.2	CU-02 — Gestionar solicitud (Admin)	14
4.4.3	CU-03 — Pagar solicitud aprobada (Cliente)	15
4.4.4	CU-04 — Registrar movimiento de stock (Empleado / Worker)	15
5	3. Diseño del sistema	16
5.1	3.1 Arquitectura general	16
5.2	3.2 Modelo de base de datos	17
5.2.1	Área Catálogo	17
5.2.2	Área Usuarios	18

5.2.3	Área Solicitudes	19
5.2.4	Área Operativa	19
5.2.5	Restricciones de integridad destacadas	20
5.3	3.3 Diseño de la API REST	20
5.3.1	Seguridad y autenticación	20
5.3.2	Estructura de endpoints	21
5.3.3	Gestión de errores	22
5.4	3.4 Diseño de interfaces	22
5.4.1	Web Shop	22
5.4.2	Aplicación Desktop (JavaFX)	22
5.4.3	Worker Móvil (React Native)	23
6	4. Implementación	24
6.1	4.1 Tecnologías utilizadas	24
6.1.1	Backend — API REST	24
6.1.2	Frontend Web — Web Shop	24
6.1.3	Desktop — JavaFX	25
6.1.4	Worker Móvil — React Native	25
6.1.5	Infraestructura y servicios externos	26
6.2	4.2 API REST — Spring Boot	26
6.2.1	Estructura del proyecto	26
6.2.2	Seguridad — JWT y Spring Security	27
6.2.3	Servicio de solicitudes y negociación	28
6.2.4	Resistencia a fallos de RabbitMQ	28
6.2.5	Endpoint de stock sin autenticación	29
6.3	4.3 Web Shop — React	29
6.3.1	Estructura de la aplicación	29
6.3.2	Gestión de estado con Zustand	30
6.3.3	Interceptores de Axios	30
6.3.4	Integración de pagos con Stripe Elements	31
6.3.5	Generación de facturas PDF con jsPDF	31
6.4	4.4 Aplicación Desktop — JavaFX	32
6.4.1	Arquitectura MVC con FXML	32
6.4.2	Módulo de solicitudes	32
6.4.3	Generación de instaladores	32
6.5	4.5 Worker Móvil — React Native	32
6.5.1	Estructura de navegación (Expo Router)	32
6.5.2	Gestión de autenticación con caché en memoria	33

6.5.3	Registro de movimientos de stock	33
6.5.4	Escáner de códigos QR	34
6.5.5	Compilación del APK	34
6.6	4.6 Integración Odoo 17	34
6.6.1	Flujo de creación de pedido de venta	34
6.6.2	Distribución proporcional del precio negociado	35
6.7	4.7 Mensajería asíncrona con RabbitMQ	35
6.7.1	Configuración del exchange	35
6.7.2	Eventos publicados	35
6.8	4.8 Pagos con Stripe	36
6.8.1	Flujo del lado del servidor (Spring Boot)	36
6.8.2	Flujo del lado del cliente (React)	36
6.8.3	Verificación server-side antes de marcar como pagada	37
7	5. Despliegue e infraestructura	38
7.1	5.1 Servidor	38
7.2	5.2 Servicios en producción	38
7.3	5.3 Servicio systemd de la API	38
7.4	5.4 Configuración de Nginx	39
7.5	5.5 Procedimiento de despliegue	40
8	6. Pruebas	41
8.1	6.1 Estrategia de pruebas	41
8.2	6.2 Pruebas de la API REST	41
8.2.1	Autenticación	41
8.2.2	Solicitudes de presupuesto	41
8.2.3	Stock	42
8.2.4	Pagos	42
8.3	6.3 Pruebas de integración	43
8.3.1	Integración con Odoo	43
8.3.2	Integración con RabbitMQ	43
8.4	6.4 Pruebas end-to-end	43
9	7. Conclusiones	45
9.1	7.1 Logros alcanzados	45
9.2	7.2 Dificultades encontradas	45
9.3	7.3 Líneas futuras	46
10	8. Bibliografía	48
10.1	Documentación oficial	48
10.2	Libros y recursos de aprendizaje	48

10.3 Herramientas y servicios utilizados	49
--	----

1. AutoCiclo

1.1 Sistema de Gestión Multiplataforma para Desguace de Vehículos

Alumno: Yalil Musa Talhaoui

Ciclo formativo: 2.º DAM — Desarrollo de Aplicaciones Multiplataforma

Centro: IES P. Hermenegildo Lanz — Granada

Curso académico: 2025 / 2026

Fecha de entrega: 19 de mayo de 2026

2. Índice

1. Introducción

- 1.1 Contexto y motivación
- 1.2 Descripción general
- 1.3 Objetivos
- 1.4 Alcance
- 1.5 Metodología

2. Análisis del sistema

- 2.1 Identificación de actores
- 2.2 Requisitos funcionales
- 2.3 Requisitos no funcionales
- 2.4 Casos de uso

3. Diseño del sistema

- 3.1 Arquitectura general
- 3.2 Modelo de base de datos
- 3.3 Diseño de la API REST
- 3.4 Diseño de interfaces

4. Implementación

- 4.1 Tecnologías utilizadas
- 4.2 API REST — Spring Boot
- 4.3 Web Shop — React
- 4.4 Aplicación Desktop — JavaFX
- 4.5 Worker Móvil — React Native
- 4.6 Integración Odoo 17
- 4.7 Mensajería asíncrona con RabbitMQ
- 4.8 Pagos con Stripe

5. Despliegue e infraestructura

6. Pruebas

7. Conclusiones

8. Bibliografía

3. 1. Introducción

3.1 1.1 Contexto y motivación

Los desguaces de vehículos son negocios complejos que deben coordinar múltiples flujos de trabajo simultáneos: la compra y catalogación de vehículos, la extracción y almacenamiento de piezas, la atención a clientes que buscan repuestos específicos, la gestión de precios negociados y el control de stock en tiempo real. Sin embargo, la mayoría de estos negocios operan con herramientas desconectadas entre sí —hojas de cálculo, sistemas de facturación independientes o incluso apuntes en papel— lo que genera ineficiencias, errores y una experiencia de cliente deficiente.

AutoCiclo nace para resolver este problema mediante un ecosistema digital integrado y multiplataforma que cubre el ciclo de vida completo de una pieza de desguace: desde que el vehículo entra al negocio hasta que la pieza llega al cliente final.

El proyecto es el resultado del trabajo de fin de ciclo del segundo año del ciclo formativo de Grado Superior en **Desarrollo de Aplicaciones Multiplataforma (DAM)**, aplicando de manera práctica las competencias adquiridas a lo largo de los dos años del ciclo: programación orientada a objetos, bases de datos relacionales, desarrollo web, desarrollo móvil, seguridad en aplicaciones y despliegue en entornos de producción.

3.2 1.2 Descripción general

AutoCiclo es un **ecosistema multiplataforma** formado por cinco componentes interconectados que comparten una única fuente de verdad:

Componente	Tecnología	Usuario destino
API REST	Spring Boot 3 + Java 21	Capa de integración
Web Shop	React 19 + TypeScript + Vite	Clientes finales
Desktop	Java 21 + JavaFX	Administradores del negocio
Worker Móvil	React Native + Expo SDK	Operarios del almacén

Componente	Tecnología	Usuario destino
Odoo 17 CE	ERP con integración JSON-RPC	Contabilidad/facturación

Todos los clientes (web, escritorio, móvil) consumen la misma API REST, que a su vez orquesta la base de datos MySQL, el broker de mensajería RabbitMQ, la pasarela de pagos Stripe y el ERP Odoo. La aplicación está completamente desplegada en un servidor Ubuntu real en la nube, accesible desde Internet.

3.3 1.3 Objetivos

3.3.1 Objetivo general

Desarrollar un sistema completo de gestión para un desguace de vehículos que integre todas las plataformas necesarias para el negocio (web, escritorio y móvil) sobre una API REST centralizada, con capacidades de pago online, integración ERP y mensajería asíncrona.

3.3.2 Objetivos específicos

1. Diseñar e implementar una **API REST** segura con autenticación JWT y control de acceso basado en roles (ADMIN, EMPLEADO, CLIENTE).
2. Desarrollar una **tienda web** que permita a los clientes explorar el catálogo, solicitar presupuestos con precio propuesto y pagar online con Stripe.
3. Implementar un **sistema de negociación multi-ronda** que permita al cliente y al administrador llegar a un acuerdo sobre el precio antes del pago.
4. Crear una **aplicación de escritorio** en JavaFX para la gestión interna del negocio, con dashboard, gestión de solicitudes e inventario.
5. Desarrollar una **app móvil** para el operario del almacén que le permita ver pedidos pendientes, escanear códigos QR de piezas y registrar movimientos de stock.
6. Integrar el sistema con **Odoo 17** para la gestión automática de pedidos de venta al confirmarse un pago.
7. Implementar **mensajería asíncrona** con RabbitMQ para eventos críticos del sistema.
8. **Desplegar** toda la infraestructura en un servidor en producción real.

3.4 1.4 Alcance

El proyecto abarca el ciclo completo de una transacción en un desguace de vehículos:

1. **Gestión del catálogo:** vehículos en el desguace y piezas disponibles con stock.
2. **Solicitudes de presupuesto:** el cliente propone su precio; no hay precio fijo.
3. **Negociación:** sistema de turnos con historial de ofertas y contraofertas.
4. **Aprobación y pago:** Stripe Elements para pago con tarjeta, verificación server-side.
5. **Preparación del pedido:** el operario recoge las piezas físicamente, el stock baja automáticamente.
6. **Facturación ERP:** Odoo crea el pedido de venta automáticamente al confirmarse el pago.
7. **Notificaciones:** sistema de notificaciones en tiempo real entre actores.

Quedan fuera del alcance: la gestión de envíos y logística externa, la integración con proveedores, y la contabilidad avanzada (que se delega a Odoo).

3.5 1.5 Metodología

El proyecto se ha desarrollado siguiendo una metodología **iterativa e incremental** adaptada al contexto académico, con las siguientes fases:

1. **Análisis y diseño** (febrero 2026): requisitos, modelo de datos, diseño de API y prototipos de UI.
2. **Desarrollo del núcleo** (marzo 2026): API REST, base de datos, autenticación JWT.
3. **Desarrollo de clientes** (abril 2026): Web Shop y aplicación Desktop en paralelo.
4. **Integración de servicios** (abril–mayo 2026): Stripe, Odoo, RabbitMQ.
5. **App móvil y APK** (mayo 2026): React Native, Worker, compilación del APK.
6. **Pruebas y despliegue** (mayo 2026): pruebas de integración, despliegue en producción.

Las herramientas de gestión utilizadas han sido GitHub para control de versiones, con commits frecuentes que documentan la evolución del proyecto.

4. 2. Análisis del sistema

4.1 2.1 Identificación de actores

El sistema tiene tres actores principales:

CLIENTE — El comprador de piezas. Se registra en la Web Shop, explora el catálogo, solicita presupuestos con su precio propuesto, negocia con el administrador y paga online. Solo ve sus propias solicitudes.

ADMIN (Administrador) — El gestor del negocio. Accede desde la Web Shop (panel admin) y desde la aplicación Desktop. Gestiona el catálogo completo, responde solicitudes, negocia precios, aprueba o rechaza solicitudes, y supervisa el stock y las estadísticas.

EMPLEADO — El operario del almacén. Accede desde la app móvil Worker. Ve los pedidos aprobados, recoge las piezas físicamente, registra movimientos de stock y consulta alertas de stock bajo. No tiene acceso a información comercial o financiera.

4.2 2.2 Requisitos funcionales

4.2.1 RF-01 — Gestión de usuarios y autenticación

- El sistema permite registro de nuevos clientes con email, nombre, teléfono, dirección y NIF.
- El sistema autentica usuarios mediante credenciales email/contraseña y devuelve un token JWT.
- Los tokens JWT tienen expiración configurable.
- El sistema controla el acceso a endpoints según el rol del usuario (ADMIN, EMPLEADO, CLIENTE).
- Los administradores pueden crear, modificar y desactivar usuarios.

4.2.2 RF-02 — Catálogo de vehículos y piezas

- El sistema mantiene un catálogo de vehículos con: matrícula, marca, modelo, año, color, fecha de entrada, estado (completo / desguazando / desguazado), precio de compra, kilometraje y ubicación GPS.

- El sistema mantiene un catálogo de piezas con: código, nombre, categoría, precio de venta, stock disponible, stock mínimo, ubicación en almacén, marcas compatibles, imagen y descripción.
- El catálogo de piezas es público y no requiere autenticación.
- Los administradores pueden crear, modificar y eliminar vehículos y piezas.
- Cada pieza puede asociarse a uno o varios vehículos mediante el inventario.

4.2.3 RF-03 — Sistema de solicitudes de presupuesto

- Los clientes pueden crear solicitudes de presupuesto seleccionando piezas del catálogo.
- Cada solicitud incluye: lista de piezas con cantidad, notas opcionales y precio ofertado por el cliente.
- El sistema asigna a cada solicitud un estado: pendiente, en_negociacion, aprobada, rechazada, pagada.
- Los administradores ven todas las solicitudes; los clientes solo las propias.

4.2.4 RF-04 — Sistema de negociación multi-ronda

- El administrador puede aprobar directamente una solicitud con el precio que estime oportuno.
- El administrador puede contraofertar: envía un precio diferente y un mensaje, el turno pasa al cliente.
- El cliente puede aceptar la contraoferta, rechazarla o hacer una nueva oferta.
- Cada movimiento queda registrado en el historial de negociación con ronda, autor, precio y fecha.
- El sistema controla el turno (quién debe actuar a continuación).

4.2.5 RF-05 — Pagos con Stripe

- Los clientes pueden pagar solicitudes en estado aprobada mediante tarjeta bancaria.
- El backend crea un PaymentIntent en Stripe y devuelve el `clientSecret` al frontend.
- El frontend confirma el pago con Stripe Elements (sin almacenar datos de tarjeta en el servidor).
- El backend verifica el estado del pago en Stripe antes de marcar la solicitud como pagada.
- Se bloquea el doble pago: solo se puede iniciar un pago sobre una solicitud en estado aprobada.

4.2.6 RF-06 — Gestión de stock

- El sistema registra movimientos de stock (entrada/salida) para cada pieza.
- Cada movimiento queda auditado con usuario, fecha, tipo y cantidad.
- El sistema detecta automáticamente piezas con `stockDisponible ≤ stockMínimo` y las expone como alertas.
- Los movimientos se registran automáticamente cuando el Worker recoge piezas de un pedido.

4.2.7 RF-07 — Códigos QR

- El sistema puede generar y almacenar códigos QR para piezas y vehículos.
- Cada QR tiene un código único que permite identificar la pieza o vehículo escaneado.
- El Worker puede escanear QR con la cámara del móvil para navegar directamente al detalle.

4.2.8 RF-08 — Notificaciones

- El sistema genera notificaciones automáticas en eventos clave: nueva solicitud, cambio de estado, pago recibido, pedido Odoo creado.
- Los usuarios pueden consultar sus notificaciones y marcarlas como leídas.

4.2.9 RF-09 — Integración Odoo

- Al confirmarse el pago de una solicitud, el sistema crea automáticamente un pedido de venta en Odoo 17.
- El pedido incluye las líneas de producto (piezas), precios negociados y datos del cliente.
- Se aplica IVA 21 % al pedido en Odoo.
- La referencia del pedido Odoo queda asociada a la solicitud.
- Si Odoo no está disponible, la operación continúa sin error.

4.2.10 RF-10 — Mensajería asíncrona (RabbitMQ)

- El sistema publica eventos en RabbitMQ al crear o modificar solicitudes.
- El sistema publica alertas de stock cuando el stock disponible baja del mínimo.
- Los errores de conexión con RabbitMQ no interrumpen las operaciones principales.

4.3 2.3 Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	Todas las contraseñas se almacenan hasheadas con BCrypt (coste 12).
RNF-02	Seguridad	Las comunicaciones entre clientes y API usan tokens JWT con algoritmo HS256.
RNF-03	Seguridad	Los datos de tarjeta bancaria nunca pasan por el servidor; se delegan a Stripe.
RNF-04	Disponibilidad	La API, la Web Shop y la base de datos están desplegadas en un servidor con disponibilidad 24/7.
RNF-05	Rendimiento	El timeout de conexión a RabbitMQ es de 2 segundos para fallo rápido.
RNF-06	Rendimiento	El timeout de la API en el Worker móvil es de 10 segundos.
RNF-07	Portabilidad	La app Desktop se distribuye en formato .deb para Linux con JRE embebido y en .zip portable para Windows.
RNF-08	Portabilidad	La app Worker se distribuye como APK de Android (arm64-v8a).
RNF-09	Mantenibilidad	La API usa Spring Boot con inyección de dependencias; cada capa (controller, service, repository) está separada.

ID	Categoría	Descripción
RNF-10	Usabilidad	La Web Shop y el Worker están optimizados para dispositivos con pantalla de escritorio y móvil respectivamente.
RNF-11	Tolerancia a fallos	Los fallos de Odoo y RabbitMQ son silenciosos: las operaciones principales completan sin error.

4.4 2.4 Casos de uso

4.4.1 CU-01 — Solicitar presupuesto (Cliente)

Actor: Cliente autenticado

Precondición: El cliente ha iniciado sesión y hay piezas disponibles en el catálogo.

Flujo principal: 1. El cliente navega al catálogo y selecciona una o varias piezas. 2. Accede a /solicitar y rellena el formulario con su precio propuesto y notas opcionales. 3. Envía la solicitud. 4. El sistema crea la solicitud en estado pendiente, registra la oferta inicial en el historial y envía una notificación a todos los administradores. 5. El cliente es redirigido a “Mis Solicitudes” donde puede ver la nueva solicitud.

Flujo alternativo:

- Si el cliente no ha iniciado sesión, es redirigido a la página de login antes de poder acceder al formulario.

4.4.2 CU-02 — Gestionar solicitud (Admin)

Actor: Administrador

Precondición: Existe al menos una solicitud en estado pendiente o en_negociacion.

Flujo principal (aprobar directamente): 1. El admin selecciona la solicitud en la lista. 2. Introduce el precio final y un mensaje de respuesta. 3. Pulsa “Aprobar”. 4. El sistema cambia el estado a aprobada, notifica al cliente y crea el pedido en Odoo si está disponible.

Flujo alternativo A (contraofertar): 1. El admin introduce un precio diferente y pulsa “Contraofertar”. 2. El sistema cambia el estado a `en_negociacion` y el turno a cliente. 3. Se notifica al cliente y se publica el evento en RabbitMQ.

Flujo alternativo B (rechazar): 1. El admin pulsa “Rechazar” con un motivo. 2. El sistema cambia el estado a `rechazada` y notifica al cliente.

4.4.3 CU-03 — Pagar solicitud aprobada (Cliente)

Actor: Cliente autenticado

Precondición: La solicitud está en estado aprobada con `precioTotal` establecido.

Flujo principal: 1. El cliente va a “Mis Solicitudes” y pulsa “Pagar” en la solicitud aprobada. 2. El frontend llama a `POST /api/pagos/intento` con el ID de la solicitud. 3. El backend crea un `PaymentIntent` en Stripe y devuelve el `clientSecret`. 4. El frontend presenta el formulario de Stripe Elements. 5. El cliente introduce sus datos de tarjeta y confirma el pago. 6. Stripe confirma el pago; el frontend llama al backend para marcar la solicitud como pagada. 7. El backend verifica el estado del `PaymentIntent` en Stripe, actualiza el estado y llama a Odoo. 8. Se genera una notificación de pago completado al cliente y al administrador.

Flujo alternativo (pago rechazado):

- Si Stripe rechaza la tarjeta, se muestra el mensaje de error de Stripe y la solicitud permanece en aprobada.

4.4.4 CU-04 — Registrar movimiento de stock (Empleado / Worker)

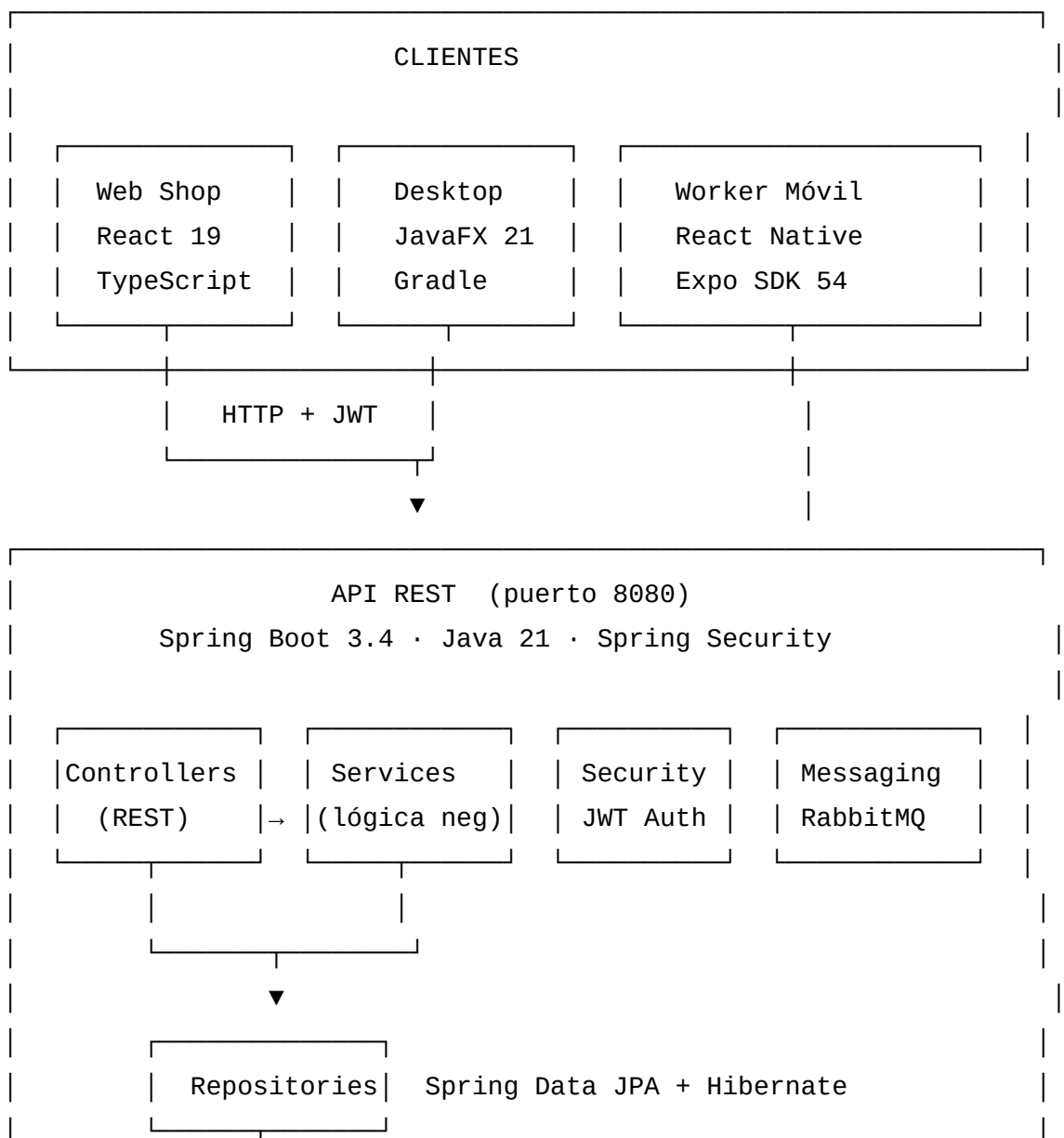
Actor: Empleado autenticado en la app Worker

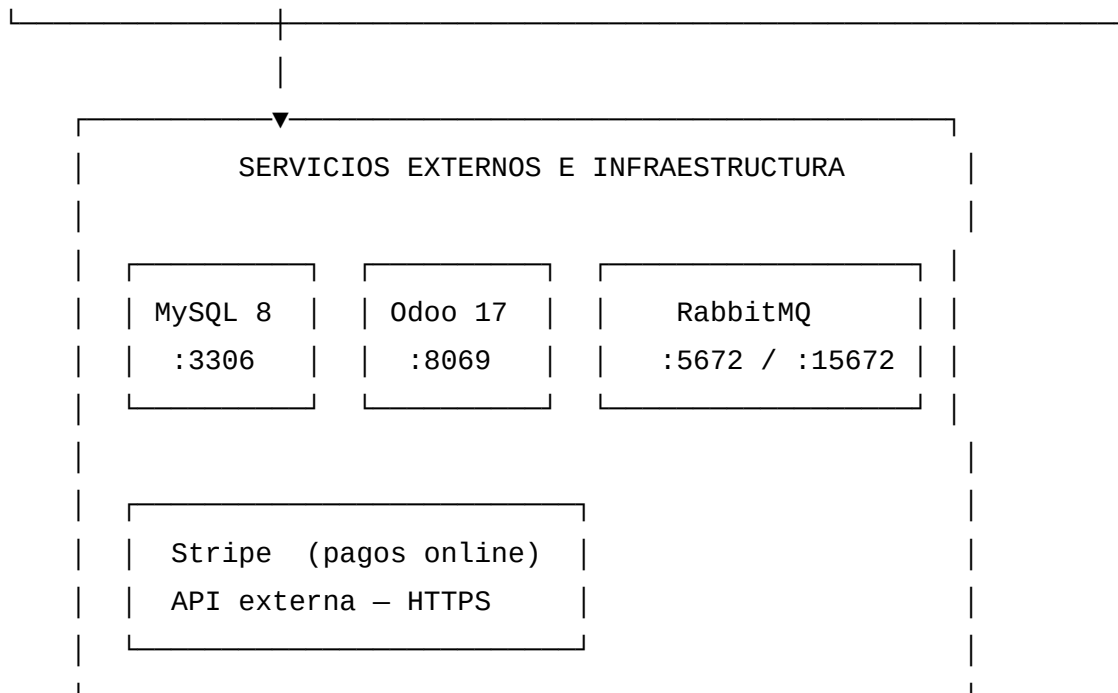
Flujo principal: 1. El empleado navega al detalle de una pieza (directo o mediante escaneo QR). 2. Selecciona el tipo de movimiento (entrada/salida) e introduce la cantidad. 3. Pulsa “Añadir stock” o “Retirar stock”. 4. El sistema llama a `POST /api/stock/movimiento` con los parámetros. 5. El stock de la pieza se actualiza en la base de datos. 6. Si el stock resultante es inferior al mínimo, se publica una alerta en RabbitMQ. 7. La pantalla del Worker muestra el stock actualizado sin necesidad de recargar.

5. 3. Diseño del sistema

5.1 3.1 Arquitectura general

AutoCiclo sigue una arquitectura **cliente-servidor multicapa** con una API REST central y tres clientes especializados. La separación de responsabilidades es clara: la API REST es la única que accede directamente a la base de datos y a los servicios externos (Odoo, Stripe, RabbitMQ); los clientes solo hablan con la API.





Nginx actúa como proxy inverso en el servidor: sirve el frontend React en el puerto 8090 y puede redirigir peticiones a la API en el puerto 8080.

5.2 3.2 Modelo de base de datos

La base de datos MySQL 8.0 contiene **12 tablas** organizadas en cuatro áreas funcionales:

5.2.1 Área Catálogo

VEHICULOS — Vehículos que han entrado al desguace.

Campo	Tipo	Descripción
id_vehiculo	INT PK	Identificador
matricula	VARCHAR(10) UNIQUE	Matrícula del vehículo
marca	VARCHAR(50)	Fabricante
modelo	VARCHAR(50)	Modelo
anio	INT	Año de fabricación
color	VARCHAR(30)	Color
fecha_entrada	DATE	Fecha de entrada al desguace

Campo	Tipo	Descripción
estado	ENUM	completo / desguazando / desguazado
precio_compra	DECIMAL(10,2)	Precio de adquisición
kilometraje	INT	Kilómetros del vehículo
ubicacion_gps	VARCHAR(50)	Localización en el patio
observaciones	TEXT	Notas libres

PIEZAS — Piezas disponibles para la venta.

Campo	Tipo	Descripción
id_pieza	INT PK	Identificador
codigo_pieza	VARCHAR(20) UNIQUE	Código interno de la pieza
nombre	VARCHAR(100)	Nombre descriptivo
categoria	ENUM	motor / carroceria / interior / electronica / ruedas / otros
precio_venta	DECIMAL(10,2)	Precio de catálogo
stock_disponible	INT	Unidades en almacén
stock_minimo	INT	Nivel mínimo para alertas
ubicacion_almacen	VARCHAR(50)	Posición en el almacén
compatible_marcas	TEXT	Marcas de vehículo compatibles
imagen	LONGTEXT	Imagen en Base64 o URL
descripcion	TEXT	Descripción técnica

INVENTARIO_PIEZAS — Relación muchos-a-muchos entre vehículos y piezas (clave compuesta `id_vehiculo` + `id_pieza`).

5.2.2 Área Usuarios

ROLES — Roles del sistema: ADMIN, EMPLEADO, CLIENTE.

USUARIOS — Cuentas de usuario con referencia al rol y hash BCrypt de la contraseña.

CLIENTES — Perfil extendido del cliente (teléfono, dirección, NIF), relacionado con USUARIOS mediante FK única.

5.2.3 Área Solicitudes

SOLICITUDES_PRESUPUESTO — Solicitud principal con el estado de la negociación.

Campo	Tipo	Descripción
id_solicitud	INT PK	Identificador
id_cliente	INT FK	Cliente que solicita
fecha_solicitud	DATETIME	Fecha de creación
estado	ENUM	pendiente / en_negociacion / aprobada / rechazada / pagada
respuesta_admin	TEXT	Mensaje del administrador
precio_total	DECIMAL	Precio final acordado
precio_oferta_cliente	DECIMAL	Última oferta del cliente
precio_contraoferta	DECIMAL	Última contraoferta del admin
turno	ENUM	cliente / admin — quién debe actuar
referencia_odoo	VARCHAR(50)	Referencia del pedido en Odoo

DETALLE_SOLICITUD — Líneas de piezas de cada solicitud (clave compuesta id_solicitud + id_pieza).

NEGOCIACION_HISTORIAL — Registro de cada ronda de negociación con autor, precio y mensaje.

5.2.4 Área Operativa

CODIGOS_QR — Códigos QR generados para piezas y vehículos con código único.

MOVIMIENTOS_STOCK — Auditoría de entradas y salidas de stock con usuario y fecha.

NOTIFICACIONES — Notificaciones para usuarios con tipo, mensaje y estado de lectura.

5.2.5 Restricciones de integridad destacadas

- La matrícula de un vehículo es única (UNIQUE KEY).
- El código de pieza es único (UNIQUE KEY).
- Un usuario puede tener exactamente un perfil de cliente (UNIQUE en id_usuario de CLIENTES).
- El charset de toda la base de datos es utf8mb4_unicode_ci para soporte de caracteres Unicode.
- Se usan ON DELETE CASCADE / ON DELETE RESTRICT apropiados en las FK para mantener la integridad referencial.

5.3 3.3 Diseño de la API REST

La API sigue los principios REST: recursos identificados por URL, uso correcto de métodos HTTP (GET, POST, PUT, DELETE) y respuestas con códigos HTTP semánticos.

5.3.1 Seguridad y autenticación

La seguridad se implementa con Spring Security y tokens JWT (JSON Web Token):

1. El cliente envía credenciales a POST /api/auth/login.
2. La API verifica las credenciales contra la base de datos (BCrypt).
3. Si son correctas, genera un JWT firmado con la clave secreta configurada en variables de entorno.
4. El cliente incluye el token en la cabecera Authorization: Bearer <token> en todas las peticiones protegidas.
5. El filtro JwtAuthFilter intercepta cada petición, valida el token y carga el UserDetails en el contexto de Spring Security.
6. Los controladores usan @PreAuthorize para restringir el acceso según el rol.

Endpoints públicos (sin autenticación): - POST /api/auth/login y POST /api/auth/register
 - GET /api/piezas/**, GET /api/vehiculos/** — catálogo público - GET /api/inventario/piezas
 GET /api/codigos-qr/** - POST /api/stock/movimiento — excepción para el Worker (sin JWT, con fallback de usuario) - POST /api/pagos/webhook — webhook de Stripe

5.3.2 Estructura de endpoints

Autenticación

POST /api/auth/login

Body: { email, password }

Resp: { token, usuario: { id, nombre, email, rol } }

POST /api/auth/register

Body: { nombre, email, password, telefono, direccion, nif }

Resp: 201 Created

Solicitudes de presupuesto

GET /api/solicitudes → ADMIN: todas; CLIENTE: las suyas

GET /api/solicitudes/{id} → Detalle con detalles e historial

POST /api/solicitudes → Crear solicitud (CLIENTE)

PUT /api/solicitudes/{id}/aprobar → ADMIN aprueba

PUT /api/solicitudes/{id}/rechazar → ADMIN rechaza

PUT /api/solicitudes/{id}/contraoferta → ADMIN contraoferta

PUT /api/solicitudes/{id}/aceptar-oferta → CLIENTE acepta

PUT /api/solicitudes/{id}/rechazar-oferta → CLIENTE rechaza

PUT /api/solicitudes/{id}/nueva-oferta → CLIENTE contraoferta

Stock

GET /api/stock/alertas → Piezas con stock ≤ mínimo (ADMIN/EMPLEADO)

GET /api/stock/movimientos/{idPieza} → Historial de movimientos (ADMIN/EMPLEADO)

POST /api/stock/movimiento → Registrar movimiento (PÚBLICO)

Body: { idPieza, tipo: 'entrada' | 'salida', cantidad, notas? }

Pagos

POST /api/pagos/intento

Body: { solicitudId }

Resp: { clientSecret, importeTotal, solicitudId }

POST /api/pagos/confirmar

Body: { solicitudId, paymentIntentId }

Piezas, Vehículos, Usuarios, Notificaciones, Códigos QR — CRUD estándar con protección por rol.

5.3.3 Gestión de errores

La clase `GlobalExceptionHandler` centraliza el manejo de excepciones y devuelve respuestas JSON uniformes:

- `IllegalArgumentException` → 400 Bad Request
- `IllegalStateException` → 409 Conflict
- Acceso no autorizado → 401 / 403
- Recurso no encontrado → 404

5.4 3.4 Diseño de interfaces

5.4.1 Web Shop

La Web Shop sigue un diseño **limpio y moderno** con Tailwind CSS, accesible desde cualquier navegador de escritorio o móvil. Las páginas principales son:

- **Home** (/): landing page con hero, categorías y llamada a la acción.
- **Catálogo** (/catalogo): grid de piezas con filtrado por categoría, búsqueda y paginación.
- **Detalle de pieza** (/catalogo/:id): información completa, stock, botón de solicitud.
- **Mis Solicitudes** (/mis-solicitudes): listado de solicitudes del cliente con badges de estado y acciones disponibles.
- **Panel de administración** (/admin): dashboard con métricas, gestión de solicitudes, piezas, vehículos y usuarios.
- **Pago** (/pagar?id=N): formulario de Stripe Elements integrado.

5.4.2 Aplicación Desktop (JavaFX)

La app Desktop sigue el patrón **MVC con FXML**: la interfaz se define en archivos FXML y los controladores Java gestionan la lógica. Las secciones principales son:

- **Login:** pantalla de autenticación con validación.
- **Dashboard / Estadísticas:** métricas del negocio en tiempo real.
- **Solicitudes:** tabla con todas las solicitudes, acciones de gestión y diálogo de historial de negociación.
- **Piezas:** CRUD completo con formularios.
- **Vehículos:** CRUD con gestión de estado.
- **Inventario:** asociación piezas-vehículos.
- **Usuarios:** gestión de cuentas (solo ADMIN).

5.4.3 Worker Móvil (React Native)

Interfaz optimizada para uso en almacén: botones grandes, iconos claros, lectura rápida. Consta de cuatro pestañas principales:

- **Dashboard:** alertas de stock con tarjetas codificadas por color (rojo = sin stock, naranja = stock bajo).
- **Pedidos:** solicitudes aprobadas pendientes de preparar.
- **Escanear:** cámara con escáner de QR en tiempo real.
- **Vehículos:** listado de vehículos del desguace.

Las pantallas secundarias incluyen detalle de pieza (con formulario de movimiento de stock) y detalle de pedido (con lista de piezas a recoger).

6. 4. Implementación

6.1 4.1 Tecnologías utilizadas

6.1.1 Backend — API REST

Tecnología	Versión	Uso
Java	21 LTS	Lenguaje de programación
Spring Boot	3.4.3	Framework web
Spring Security	6.x	Autenticación y autorización
Spring Data JPA	3.x	Acceso a datos (ORM)
Hibernate	6.x	Implementación JPA
MySQL Connector/J	8.x	Driver de base de datos
JWT (io.jsonwebtoken)	0.12.6	Generación y validación de JWT
Spring AMQP	3.x	Integración con RabbitMQ
Stripe Java SDK	25.3.0	Pasarela de pagos
Lombok	1.x	Reducción de boilerplate
Maven	3.x	Gestión de dependencias y build

6.1.2 Frontend Web — Web Shop

Tecnología	Versión	Uso
React	19.2.5	Biblioteca UI
TypeScript	6.0	Tipado estático
Vite	8.x	Bundler y servidor de desarrollo
Tailwind CSS	4.x	Estilos utilitarios
React Router DOM	7.x	Enrutamiento SPA

Tecnología	Versión	Uso
Zustand	5.x	Gestión de estado global
Axios	1.x	Cliente HTTP
Stripe React / Stripe.js	6.x / 9.x	Integración de pagos
jsPDF + jsPDF-AutoTable	4.x / 5.x	Generación de facturas PDF
Lucide React	1.x	Iconografía
Motion	12.x	Animaciones

6.1.3 Desktop — JavaFX

Tecnología	Versión	Uso
Java	21 LTS	Lenguaje de programación
JavaFX	21.0.5	Framework de interfaz gráfica
Gradle	8.x	Build y gestión de dependencias
HikariCP	5.1	Pool de conexiones
AMQP Client (RabbitMQ)	5.20	Mensajería
Gson	2.10.1	Serialización JSON
Ikonli (MaterialDesign2)	12.3.1	Iconografía
jpackage	JDK 21	Generación de instaladores

6.1.4 Worker Móvil — React Native

Tecnología	Versión	Uso
React Native	0.81.5	Framework móvil
Expo SDK	54	Toolchain y módulos nativos
Expo Router	6.x	Navegación basada en archivos
NativeWind	latest	Tailwind para React Native

Tecnología	Versión	Uso
Zustand	4.x	Estado global
Axios	1.x	Cliente HTTP
expo-secure-store	15.x	Almacenamiento seguro del JWT
expo-camera	17.x	Acceso a la cámara
qrcode	1.x	Generación de QR

6.1.5 Infraestructura y servicios externos

Servicio	Versión	Uso
Ubuntu Server	22.04 LTS	Sistema operativo del servidor
MySQL	8.0	Base de datos relacional
Nginx	1.x	Servidor web / proxy inverso
RabbitMQ	3.x	Broker de mensajería AMQP
Odoo	17 CE	ERP (facturación y pedidos)
Stripe	API v1	Pagos con tarjeta online
Contabo	—	Proveedor VPS (Ubuntu Server, 8 GB RAM, 150 GB SSD)

6.2 4.2 API REST — Spring Boot

6.2.1 Estructura del proyecto

API/autociclo-api/src/main/java/com/autociclo/

```

├─ AutocicloApiApplication.java      ← Punto de entrada
├─ config/
│   ├─ SecurityConfig.java          ← Configuración Spring Security + CORS
│   ├─ RabbitMQConfig.java          ← Exchange, colas y bindings
│   └─ GlobalExceptionHandler.java ← Manejo centralizado de excepciones
├─ controllers/                     ← Capa HTTP (REST)
│   ├─ AuthController.java
│   └─ PiezaController.java

```

		VehiculoController.java	
		InventarioController.java	
		SolicitudController.java	
		StockController.java	
		PagoController.java	
		UsuarioController.java	
		CodigoQRController.java	
		NotificacionController.java	
		services/	← Lógica de negocio
		AuthService.java	
		SolicitudService.java	← Negociación + Odoo + Stripe
		StockService.java	
		PiezaService.java	
		VehiculoService.java	
		InventarioService.java	
		UsuarioService.java	
		CodigoQRService.java	
		NotificacionService.java	
		repositories/	← Spring Data JPA (interfaces)
		models/	← Entidades JPA
		dto/	← Objetos de transferencia de datos
		security/	
		JwtUtil.java	← Generación y validación de JWT
		JwtAuthFilter.java	← Filtro de autenticación por token
		UserDetailsServiceImpl.java	← Carga de usuario desde BD
		messaging/	
		RabbitMQPublisher.java	← Publicación de eventos
		RabbitMQConsumer.java	← Consumo de eventos
		utils/	
		OdooClient.java	← Cliente JSON-RPC para Odoo 17

6.2.2 Seguridad — JWT y Spring Security

La configuración de seguridad (SecurityConfig.java) define:

1. **Stateless session management:** no hay sesiones en servidor; cada petición se autentica por JWT.
2. **CORS configurado:** permite peticiones desde los orígenes de los clientes web.

3. **Lista de rutas públicas:** catálogo de piezas, vehículos, autenticación y webhook de Stripe.
4. **Filtro JWT** (JwtAuthFilter) insertado antes de UsernamePasswordAuthenticationFilter.

El filtro JWT funciona así:

```
// Extrae el token de la cabecera Authorization: Bearer <token>
// Si el token es válido → carga el UserDetails y establece el contexto de seguridad
// Si no hay token o es inválido → la petición continúa sin autenticación
// (será rechazada por el filtro de acceso si el endpoint requiere autenticación)
```

La clase JwtUtil usa la librería JJWT 0.12.x con firma HMAC-SHA256 y expiración configurable.

6.2.3 Servicio de solicitudes y negociación

SolicitudService.java es el núcleo del negocio. Implementa la máquina de estados de una solicitud:

```
pendiente —(admin contraoferta)—→ en_negociacion
                                     |
pendiente —(admin aprueba)————→ aprobada
pendiente —(admin rechaza)————→ rechazada
en_negociacion —(cliente acepta)——→ aprobada
en_negociacion —(cliente rechaza)——→ rechazada
en_negociacion —(cliente contraoferta)→ en_negociacion (turno vuelve a admin)
aprobada —(cliente paga)————→ pagada
pagada —(empleado envía)————→ enviado
```

Cada transición de estado registra una entrada en NEGOCIACION_HISTORIAL y envía notificaciones.

6.2.4 Resistencia a fallos de RabbitMQ

Todos los métodos de RabbitMQPublisher están envueltos en bloques try-catch:

```
public void publicarNuevaSolicitud(int idSolicitud, String nombreCliente, String email)
{
    try {
        rabbitTemplate.convertAndSend(
            "autociclo.exchange", "solicitudes.nueva",
            Map.of("idSolicitud", idSolicitud, "cliente", nombreCliente)
        );
    }
}
```

```

    } catch (Exception e) {
        log.warn("RabbitMQ no disponible al publicar solicitud {}: {}", idSolicitud, e);
    }
}

```

Esto garantiza que si RabbitMQ no está disponible, las operaciones principales (crear solicitud, aprobar, etc.) continúan sin interrupción.

6.2.5 Endpoint de stock sin autenticación

Para facilitar el uso del Worker móvil en entornos de demostración, el endpoint POST `/api/stock/movimiento` es público. Cuando se llama sin token, el servicio usa `empleado@autociclo.com` como usuario de auditoría por defecto:

```

// StockController.java
@PostMapping("/movimiento")
public ResponseEntity<MovimientoStock> registrar(
    @Valid @RequestBody MovimientoStockRequest req,
    @AuthenticationPrincipal UserDetails userDetails) {
    String email = userDetails != null
        ? userDetails.getUsername()
        : "empleado@autociclo.com";
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(stockService.registrarMovimiento(req, email));
}

```

6.3 4.3 Web Shop — React

6.3.1 Estructura de la aplicación

Web/autociclo-shop/src/

— App.tsx	← Enrutador principal
— api/	
— client.ts	← Axios + interceptores JWT + manejo de 401
— store/	
— authStore.ts	← Zustand: token + usuario (persistido en localStorage)
— carritoStore.ts	← Estado del carrito
— pages/	
— Home.tsx	
— Catalogo.tsx	← Grid de piezas + filtros

```

|   └─ DetallePieza.tsx          ← Detalle + botón solicitar
|   └─ Login.tsx / Registro.tsx
|   └─ SolicitarPresupuesto.tsx ← Formulario de solicitud
|   └─ MisSolicitudes.tsx       ← Listado + acciones + botón pagar
|   └─ Pago.tsx                 ← Stripe Elements
|   └─ admin/
|       └─ AdminDashboard.tsx
|       └─ AdminSolicitudes.tsx ← Gestión + contraoferta + aprobar
|       └─ AdminPiezas.tsx
|       └─ AdminVehiculos.tsx
|       └─ AdminUsuarios.tsx
└─ components/
    └─ Navbar.tsx / Footer.tsx
    └─ PiezaCard.tsx
    └─ PrivateRoute.tsx          ← Redirige a /login si no autenticado
    └─ AdminRoute.tsx            ← Redirige si no es ADMIN
    └─ ClienteRoute.tsx

```

6.3.2 Gestión de estado con Zustand

El store de autenticación persiste el token y los datos del usuario en `localStorage`, lo que permite que la sesión sobreviva a recargas de página:

```

// authStore.ts (simplificado)
const useAuthStore = create(
  persist(
    (set) => ({
      token: null,
      user: null,
      login: (token, user) => set({ token, user }),
      logout: () => set({ token: null, user: null }),
    }),
    { name: 'autociclo-auth' }
  )
);

```

6.3.3 Interceptores de Axios

El cliente HTTP central adjunta automáticamente el JWT en cada petición y gestiona los errores 401 (token expirado) redirigiendo al login:

```
// api/client.ts (simplificado)
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (r) => r,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

6.3.4 Integración de pagos con Stripe Elements

La página de pago carga Stripe Elements con el `clientSecret` recibido de la API. Stripe Elements maneja el formulario de tarjeta de forma segura sin que los datos pasen por el servidor de AutoCiclo:

```
// Pago.tsx (flujo simplificado)
// 1. Llama a la API para obtener clientSecret
const { clientSecret } = await api.post('/api/pagos/intento', { solicitudId });
// 2. Muestra <Elements> con el clientSecret
// 3. Al confirmar, Stripe confirma el PaymentIntent
const { paymentIntent } = await stripe.confirmCardPayment(clientSecret, { ... });
// 4. Informa al backend para marcar la solicitud como 'pagada'
await api.post('/api/pagos/confirmar', { solicitudId, paymentIntentId: paymentIntent.id });
```

6.3.5 Generación de facturas PDF con jsPDF

Al confirmarse el pago, la Web Shop genera automáticamente una factura PDF descargable usando la librería jsPDF con el plugin jsPDF-AutoTable. La factura incluye los datos del cliente, las piezas, el precio total y la referencia de Odoo si está disponible.

6.4 4.4 Aplicación Desktop — JavaFX

6.4.1 Arquitectura MVC con FXML

La aplicación Desktop sigue el patrón MVC de JavaFX: cada vista tiene un archivo FXML (diseño) y un controlador Java (lógica). Las vistas se cargan con `FXMLLoader` y se navega entre ellas intercambiando el contenido del panel principal.

La comunicación con la API se centraliza en `ApiClient.java`, que implementa un wrapper sobre `HttpClient` de Java 11+ con soporte para adjuntar el token JWT en las cabeceras.

6.4.2 Módulo de solicitudes

`SolicitudesController.java` es el módulo más complejo del Desktop:

- Muestra todas las solicitudes en un `TableView` con columnas: ID, Cliente, Estado, Fecha, Oferta, Contraoferta, Referencia Odoo y Turno.
- El turno se formatea con indicadores visuales: “Pendiente”, “Tu turno”, “Turno del cliente”, “Cerrado”, “Cerrada”.
- Al hacer doble clic en una solicitud, se abre un diálogo (`Dialog`) de 620×500 px con el historial de negociación en formato de burbujas de chat.
- El polling REST cada 30 segundos detecta nuevas solicitudes automáticamente.

6.4.3 Generación de instaladores

La aplicación Desktop se distribuye en dos formatos:

Instalador Linux .deb (generado con `jpackage`): - Embebe el JRE de OpenJDK 21, por lo que no requiere Java instalado. - Se registra como aplicación del sistema con acceso directo. - Generado con la tarea Gradle `instaladorLinux`.

ZIP portable para Windows: - Incluye el JAR, las dependencias y el JRE de Windows 21 (Adoptium Temurin). - Se ejecuta con un script `.bat` que apunta al JRE embebido. - Generado con la tarea Gradle `portableWindows`.

6.5 4.5 Worker Móvil — React Native

6.5.1 Estructura de navegación (Expo Router)

Expo Router usa el sistema de archivos para definir las rutas, similar a Next.js:

app/	
├─ _layout.tsx	← Layout raíz + verificación de sesión
├─ login.tsx	← Pantalla de login
├─ (tabs)/	
│ └─ _layout.tsx	← Barra de pestañas (Tab Navigator)
│ └─ dashboard.tsx	← Alertas de stock
│ └─ pedidos.tsx	← Solicitudes aprobadas
│ └─ escanear.tsx	← Cámara + escáner QR
│ └─ vehiculos.tsx	← Listado de vehículos
├─ pieza/[id].tsx	← Detalle de pieza + movimiento de stock
├─ solicitud/[id].tsx	← Detalle de pedido (preparar)
└─ vehiculo/[id].tsx	← Detalle de vehículo

6.5.2 Gestión de autenticación con caché en memoria

El Worker implementa un sistema de caché en memoria para el JWT que resuelve el problema de la asincronía en los interceptores de Axios:

```
// lib/auth.ts
let _memToken: string | null = null;

// Al hacer login, el token se guarda en SecureStore (cifrado) Y en _memToken
// El interceptor de Axios lee _memToken de forma síncrona (no async)
// Si la app se reinicia, _memToken = null, pero SecureStore permite recuperarlo

export function getTokenSync(): string | null { return _memToken; }
export function setMemToken(t: string) { _memToken = t; }
export function isTokenExpired(token: string): boolean {
  const [, payload] = token.split('.');
  const { exp } = JSON.parse(atob(payload));
  return Date.now() / 1000 > exp;
}
```

6.5.3 Registro de movimientos de stock

La pantalla de detalle de pieza (pieza/[id].tsx) permite registrar entradas y salidas de stock directamente:

1. El usuario selecciona tipo (entrada/salida) e introduce la cantidad.
2. Al pulsar el botón, se valida la cantidad y se llama directamente a POST /api/stock/movimiento.

3. En caso de éxito, el stock se actualiza localmente en el estado de React sin necesidad de recargar la pieza desde la API (evita posibles errores de red secundarios).
4. En caso de error de autenticación (AUTH_REDIRECT), el interceptor de Axios gestiona el logout automático.

6.5.4 Escáner de códigos QR

La pestaña “Escanear” usa expo-camera para acceder a la cámara del dispositivo. Al detectar un QR:

1. Se consulta la API con el código único escaneado: GET /api/codigos-qr/{codigo}.
2. Si el QR es de tipo pieza, navega a /pieza/{idReferencia}.
3. Si es de tipo vehiculo, navega a /vehiculo/{idReferencia}.
4. Si el QR no existe o es inválido, muestra un mensaje de error y permite volver a escanear.

6.5.5 Compilación del APK

El APK se genera con Gradle nativo de Android (sin EAS Cloud):

```
./android/gradlew assembleDebug --project-dir android -x lint
# APK de salida: android/app/build/outputs/apk/debug/app-debug.apk
# Tamaño: ~200 MB (arm64-v8a, bundle JS embebido)
```

La opción `debuggableVariants = []` en la configuración de Metro hace que el bundle JavaScript quede embebido en el APK, por lo que no es necesario tener Metro corriendo para ejecutar la app en el dispositivo.

6.6 4.6 Integración Odoo 17

La integración con Odoo se realiza mediante el protocolo **JSON-RPC 2.0**, que es la API nativa de Odoo. El cliente `OdooClient.java` encapsula toda la comunicación:

6.6.1 Flujo de creación de pedido de venta

1. **Autenticación:** POST /web/session/authenticate → devuelve uid (identificador de sesión).
2. **Buscar o crear partner:** `res.partner.search(email)` → si no existe, `res.partner.create(...)`
3. **Buscar o crear producto:** `product.product.search(nombre)` → si no existe, crea el producto de tipo “consumible”.

4. **Crear pedido de venta:** `sale.order.create(partner_id, order_lines)` con IVA 21 % por línea.
5. **Confirmar pedido:** `sale.order.action_confirm(orderId)` → pasa el pedido a estado “sale”.
6. **Obtener referencia:** `sale.order.read(orderId, fields=['name'])` → devuelve “S00042”.

6.6.2 Distribución proporcional del precio negociado

Cuando el cliente paga un precio negociado diferente al de catálogo, los precios de cada línea del pedido de Odoo se distribuyen proporcionalmente:

```
// totalConIva = precio negociado
// baseImponible = totalConIva / 1.21 (se excluye el IVA para que Odoo lo añada)
// factor = baseImponible / sumaDePreciosCatalogo
// precioLinea_i = precioCatalogo_i * cantidad_i * factor
```

Esto asegura que el total del pedido en Odoo cuadre exactamente con lo que pagó el cliente.

6.7 4.7 Mensajería asíncrona con RabbitMQ

RabbitMQ actúa como broker de eventos para notificaciones asíncronas entre componentes.

6.7.1 Configuración del exchange

Exchange: `autociclo.exchange` (tipo: `topic`)
 Cola: `solicitudes.nueva` → routing key: `"solicitudes.nueva"`
 Cola: `stock.alerta` → routing key: `"stock.alerta"`

6.7.2 Eventos publicados

Evento	Momento	Datos
<code>solicitudes.nueva</code>	Cliente crea solicitud	<code>{idSolicitud, cliente, email}</code>
<code>solicitudes.nueva</code>	Cliente contraoferta	<code>{idSolicitud, cliente, email}</code>
<code>solicitudes.nueva</code>	Admin contraoferta	<code>{idSolicitud, cliente, precio}</code>

Evento	Momento	Datos
stock.alerta	Stock baja del mínimo	{idPieza, nombre, stockActual, stockMinimo}

Nota técnica importante: El Desktop no recibe mensajes de RabbitMQ directamente por AMQP push. En su lugar, implementa **polling REST** cada 30 segundos hacia `/api/solicitudes`. Esto simplifica la arquitectura del cliente Desktop y lo hace más robusto ante posibles desconexiones del broker.

6.8 4.8 Pagos con Stripe

La integración de pagos usa el flujo **PaymentIntent** de Stripe, que es el método recomendado para pagos con tarjeta:

6.8.1 Flujo del lado del servidor (Spring Boot)

```
// PagoController.java → POST /api/pagos/intento
Stripe.apiKey = stripeSecretKey; // clave secreta de Stripe
PaymentIntentCreateParams params = PaymentIntentCreateParams.builder()
    .setAmount((long)(solicitud.getPrecioTotal().doubleValue() * 100)) // en céntimos
    .setCurrency("eur")
    .setAutomaticPaymentMethods(...)
    .putMetadata("solicitudId", String.valueOf(solicitudId))
    .build();
PaymentIntent intent = PaymentIntent.create(params);
return Map.of("clientSecret", intent.getClientSecret(), ...);
```

6.8.2 Flujo del lado del cliente (React)

1. `loadStripe(publishableKey)` — carga Stripe.js de forma asíncrona.
2. `<Elements stripe={stripePromise} options={{ clientSecret }}>` — monta el contexto de Stripe.
3. `<PaymentElement>` — renderiza el formulario seguro de Stripe.
4. `stripe.confirmPayment({ elements, redirect: 'if_required' })` — confirma el pago.

6.8.3 Verificación server-side antes de marcar como pagada

Antes de cambiar el estado de la solicitud a pagada, el backend verifica el estado del `PaymentIntent` directamente con la API de Stripe:

```
PaymentIntent intent = PaymentIntent.retrieve(paymentIntentId);  
if (!"succeeded".equals(intent.getStatus())) {  
    throw new IllegalStateException("El pago no está confirmado en Stripe");  
}
```

Esto previene cualquier intento de marcar solicitudes como pagadas sin que el pago haya sido realmente procesado.

7. 5. Despliegue e infraestructura

7.1 5.1 Servidor

El sistema está desplegado en un VPS (Virtual Private Server) de **Contabo** con las siguientes características:

Parámetro	Valor
Sistema operativo	Ubuntu Server 22.04 LTS
IP pública	109.123.247.31
RAM	8 GB
Almacenamiento	150 GB SSD
Acceso	SSH — root

7.2 5.2 Servicios en producción

Servicio	Puerto	Descripción
MySQL 8.0	3306	Base de datos (acceso local)
Spring Boot API	8080	Servicio systemd autociclo-api
Nginx (Web Shop)	8090	Sirve el build de React
Odoo 17 CE	8069	ERP integrado
RabbitMQ	5672	Broker AMQP
RabbitMQ Management	15672	Panel de administración web

7.3 5.3 Servicio systemd de la API

La API Spring Boot se gestiona como un servicio systemd, lo que garantiza que se reinicie automáticamente si el proceso cae:

```
# /etc/systemd/system/autociclo-api.service
[Unit]
Description=AutoCiclo API REST
After=network.target mysql.service

[Service]
Type=simple
ExecStart=/usr/bin/java -jar /opt/autociclo/autociclo-api.jar
Restart=on-failure
RestartSec=10

Environment="DB_HOST=localhost"
Environment="DB_USER=autociclo"
Environment="DB_PASS=autociclo1234"
Environment="JWT_SECRET=autociclo-secret-key-minimo-32-caracteres"
Environment="STRIPE_SECRET_KEY=sk_test_..."
Environment="RABBITMQ_HOST=localhost"
Environment="ODOO_URL=http://localhost:8069"
Environment="spring.rabbitmq.connection-timeout=2000"

[Install]
WantedBy=multi-user.target
```

El uso de variables de entorno para credenciales sigue las buenas prácticas de seguridad: ninguna credencial queda hardcodeada en el código fuente.

7.4 5.4 Configuración de Nginx

Nginx actúa como servidor web para el frontend React y como proxy inverso para la API:

```
# Servir el frontend React (build estático)
server {
    listen 8090;
    root /var/www/autociclo-shop;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```


La directiva `try_files $uri $uri/ /index.html` es esencial para que el enrutamiento del lado del cliente (React Router) funcione correctamente al acceder directamente a rutas como `/catalogo` o `/admin`.

7.5 5.5 Procedimiento de despliegue

Despliegue de la API (Spring Boot):

```
# 1. Compilar (en local)
mvn package -DskipTests -q

# 2. Subir JAR al servidor
scp target/autociclo-api-1.0.0.jar root@<servidor>:/opt/autociclo/autociclo-api.jar

# 3. Reiniciar servicio
ssh root@<servidor> "systemctl restart autociclo-api"
```

Despliegue del frontend (React):

```
# 1. Compilar (en local)
npm run build

# 2. Subir build al servidor
scp -r dist root@<servidor>:/var/www/autociclo-shop/

# 3. Copiar archivos al directorio raíz de Nginx
ssh root@<servidor> "cp -r /var/www/autociclo-shop/dist/* /var/www/autociclo-shop/"
```

8. 6. Pruebas

8.1 6.1 Estrategia de pruebas

La estrategia de pruebas del proyecto contempla cuatro niveles:

1. **Pruebas de la API** (aisladas): verificación de cada endpoint con curl y Postman.
2. **Pruebas de componentes** (aisladas): verificación visual e interactiva de cada aplicación por separado.
3. **Pruebas de integración**: verificación de las integraciones externas (Stripe, Odoo, RabbitMQ).
4. **Pruebas end-to-end**: recorrido completo del flujo de negocio con todos los sistemas activos.

La colección Postman completa está disponible en docs/POSTMAN_COLLECTION.json.

8.2 6.2 Pruebas de la API REST

8.2.1 Autenticación

Prueba	Entrada	Resultado esperado	Resultado obtenido
Login correcto (admin)	email/password válidos	200 + JWT	[OK]
Login incorrecto	password equivocado	401 Unauthorized	[OK]
Acceso protegido sin token	GET /api/solicitudes sin JWT	403 Forbidden	[OK]
Acceso con rol insuficiente	Cliente intenta GET /api/usuarios	403 Forbidden	[OK]

8.2.2 Solicitudes de presupuesto

Prueba	Resultado esperado	Resultado obtenido
Crear solicitud (cliente)	201 Created, estado pendiente	[OK]
Admin aprueba solicitud	estado → aprobada, notificación al cliente	[OK]
Admin contraoferta	estado → en_negociacion, turno → cliente	[OK]
Cliente acepta contraoferta	estado → aprobada, precioTotal = contraoferta	[OK]
Operación en estado incorrecto	IllegalStateException → 409 Conflict	[OK]

8.2.3 Stock

Prueba	Resultado esperado	Resultado obtenido
Registrar salida sin token	201 Created (endpoint público)	[OK]
Salida con cantidad mayor al stock	400 Bad Request	[OK]
Stock llega a mínimo	Alerta publicada en RabbitMQ	[OK]

8.2.4 Pagos

Prueba	Resultado esperado	Resultado obtenido
Crear PaymentIntent (solicitud aprobada)	200 + clientSecret	[OK]
Crear PaymentIntent (solicitud pendiente)	409 Conflict	[OK]
Confirmar pago con tarjeta 4242...	solicitud → pagada, pedido en Odoo	[OK]
Confirmar pago con tarjeta sin fondos	error Stripe devuelto al cliente	[OK]

Prueba	Resultado esperado	Resultado obtenido
Doble pago (solicitud ya pagada)	409 Conflict	[OK]

8.3 6.3 Pruebas de integración

8.3.1 Integración con Odoo

Escenario	Resultado
Odoo disponible + solicitud aprobada	Pedido creado en Odoo, referencia guardada
Odoo no disponible	API responde correctamente, <code>referenciaOdoo = null</code>
IVA 21 % en líneas del pedido	Pedido en Odoo muestra IVA 21 %

8.3.2 Integración con RabbitMQ

Escenario	Resultado
RabbitMQ disponible	Eventos publicados y visibles en el panel de gestión
RabbitMQ no disponible	API responde sin error (try-catch silencioso)
Tiempo de conexión > 2 s	Timeout rápido, sin bloqueo de la transacción

8.4 6.4 Pruebas end-to-end

Se realizaron tres pasadas completas del flujo de negocio con todos los sistemas activos simultáneamente:

Flujo probado: 1. Cliente se registra en la Web Shop. 2. Cliente solicita presupuesto para una pieza con su precio propuesto. 3. Desktop detecta la nueva solicitud en el polling de 30 segundos. 4. Admin contraoferta desde el Desktop. 5. Cliente recibe la contraoferta en la Web Shop y acepta. 6. Admin aprueba la solicitud → Odoo crea pedido de venta. 7. Cliente

paga con tarjeta de prueba 4242 4242 4242 4242. 8. Estado cambia a pagada. Se genera la factura PDF. 9. Worker detecta el nuevo pedido aprobado en el dashboard. 10. Empleado escanea el QR de la pieza, entra al detalle y registra la salida de stock. 11. Stock baja en la base de datos; si llega al mínimo, aparece alerta en el Worker. 12. Desktop muestra la solicitud con estado pagada → no permite rechazarla.

Resultado: [OK] Flujo completo verificado en las tres pasadas.

9. 7. Conclusiones

9.1 7.1 Logros alcanzados

El proyecto AutoCiclo ha alcanzado todos los objetivos planteados en la fase de análisis. Se ha desarrollado un ecosistema multiplataforma funcional y desplegado en producción que cubre el ciclo completo de negocio de un desguace de vehículos.

Los principales logros técnicos son:

- **Arquitectura sólida:** una API REST central que coordina cuatro tecnologías distintas sin acoplamiento directo entre ellas.
- **Sistema de negociación único:** la máquina de estados con turnos y historial auditado es una funcionalidad diferencial respecto a una tienda tradicional.
- **Despliegue real:** el sistema no corre en localhost; está accesible desde Internet en un servidor de producción real.
- **Tolerancia a fallos:** los servicios externos (Odoo, RabbitMQ) pueden fallar sin que el flujo principal se interrumpa.
- **Multiplataforma real:** cuatro entornos distintos (web, escritorio, móvil Android, API Java) desarrollados e integrados de forma coherente.
- **Seguridad correcta:** JWT, BCrypt, verificación server-side de pagos, roles y permisos granulares.
- **APK funcional:** la app móvil se compila como APK nativo instalable en Android, con el bundle JS embebido.

9.2 7.2 Dificultades encontradas

Las principales dificultades técnicas durante el desarrollo fueron:

Sincronización de JWT en React Native: el sistema de interceptores de Axios requiere acceso síncrono al token, pero expo-secure-store es asíncrono. La solución fue implementar una caché en memoria (_memToken) que se popula al cargar la app y se mantiene durante la sesión.

Problema de lambda con variables no efectivamente finales en Java: al construir lambdas en los controladores JavaFX que capturaban valores de try-catch, el compilador rechazaba

la variable porque no era `effectively final`. La solución fue usar una variable `final` auxiliar:

```
double _pa = 0.0;
try { _pa = json.get("precio").getAsDouble(); } catch (Exception e) {}
final double precioAcordado = _pa;
```

Compatibilidad de caracteres BCrypt en shell: los hashes BCrypt contienen el carácter \$, que el shell de Linux interpreta como inicio de variable. Al ejecutar SQL por SSH, los heredocs con comillas simples (<< 'ENDSQL ') resolvieron el problema.

Distribución proporcional de precios en Odoo: cuando el precio negociado difiere del catálogo, era necesario distribuir el precio entre las líneas del pedido de Odoo de forma que el total con IVA cuadrara exactamente. Esto requirió dividir el total por 1,21 (base imponible) antes de distribuir proporcionalmente.

APK con cleartext traffic: Android bloquea el tráfico HTTP en APKs de debug por defecto. La configuración heredada de Expo prebuild incluye `android:usesCleartextTraffic="true"` en el manifiesto, lo que permitió el funcionamiento sin HTTPS durante el desarrollo.

9.3 7.3 Líneas futuras

Las áreas de mejora y extensión más relevantes para una versión futura serían:

1. **HTTPS en todos los servicios:** migrar a TLS con Let's Encrypt para cifrar el tráfico entre clientes y servidor.
2. **Notificaciones push en la app móvil:** usar Firebase Cloud Messaging (FCM) para notificaciones en tiempo real cuando lleguen nuevos pedidos o alertas de stock.
3. **Panel de estadísticas avanzado:** gráficos de ventas, análisis de piezas más solicitadas, predicción de stock.
4. **Búsqueda avanzada:** integrar Elasticsearch para búsqueda de texto completo en el catálogo de piezas.
5. **Autenticación OAuth2:** permitir el registro e inicio de sesión con Google o Apple.
6. **Tests automatizados:** añadir tests unitarios con JUnit 5 y Mockito para los servicios, y tests de integración con Spring Boot Test y Testcontainers.
7. **CI/CD:** configurar GitHub Actions para automatizar el build, tests y despliegue en cada push a la rama principal.
8. **Versión iOS:** la app Worker está construida con React Native y Expo, lo que facilita la compilación para iOS una vez se disponga de un Mac con Xcode.
9. **Gestión de envíos:** integrar con APIs de empresas de mensajería (MRW, SEUR) para

generar etiquetas de envío automáticamente.

10. **WebSockets:** sustituir el polling REST del Desktop por WebSockets para actualizaciones verdaderamente en tiempo real.

10. 8. Bibliografía

10.1 Documentación oficial

- **Spring Boot** — Reference Documentation 3.4.x
<https://docs.spring.io/spring-boot/docs/3.4.x/reference/html/>
- **Spring Security** — Reference Documentation 6.x
<https://docs.spring.io/spring-security/reference/>
- **React** — Documentación oficial v19
<https://react.dev>
- **React Native** — Documentación oficial
<https://reactnative.dev/docs/getting-started>
- **Expo** — Documentación SDK 54
<https://docs.expo.dev>
- **JavaFX** — OpenJFX 21 Documentation
<https://openjfx.io/javadoc/21/>
- **Stripe API** — PaymentIntents Guide
<https://stripe.com/docs/payments/payment-intents>
- **Odoo 17** — Developer API (JSON-RPC)
<https://www.odoo.com/documentation/17.0/developer/api.html>
- **RabbitMQ** — Tutorials and Guides
<https://www.rabbitmq.com/tutorials>
- **JWT** — JSON Web Token for Java
<https://github.com/jwt/jwt>

10.2 Libros y recursos de aprendizaje

- Craig Walls — *Spring in Action*, 6.ª edición. Manning Publications, 2022.
- Alex Banks, Eve Porcello — *Learning React*, 2.ª edición. O'Reilly Media, 2020.
- Kishanthan Wagenaar — *Full-Stack React, TypeScript, and Node*. Packt Publishing, 2020.

10.3 Herramientas y servicios utilizados

- **Contabo VPS** — Proveedor de infraestructura en la nube: <https://contabo.com>
- **Gradle Build Tool** — <https://gradle.org>
- **Maven** — <https://maven.apache.org>
- **Vite** — <https://vitejs.dev>
- **Zustand** — <https://zustand-demo.pmnd.rs>
- **Tailwind CSS** — <https://tailwindcss.com>
- **Axios** — <https://axios-http.com>
- **jsPDF** — <https://github.com/parallax/jsPDF>
- **NativeWind** — <https://www.nativewind.dev>
- **Adoptium Temurin** (JRE embebido para Windows) — <https://adoptium.net>
- **jpakeage** (empaquetado Java) — incluido en JDK 14+

Documentación del Trabajo de Fin de Ciclo — AutoCiclo

Yalil Musa Talhaoui · IES P. Hermenegildo Lanz · Granada · Mayo 2026